

两届TCH之后 —— AI 渗透测试 Agent 的 Harness 工程演进、防御与我的思考

原创 I3yx 淚笑的赛博日记-起零衍迹实验室 2026年5月27日 00:17 浙江

前言

半年前，我借腾讯云黑客松智能渗透挑战赛的契机，写过一篇 AI for 安全攻防的文章：
<https://mp.weixin.qq.com/s/jT4poWZ4Gfu3faXvul07HA>。但模型能力和 Agent 工程发展非常之快。现在回头看，那篇文章不少想法已经显得不够成熟，很多设计放到今天需要重新审视。

这篇文章不是否定过去，也无法给出某种未来的标准答案。我更想借两届智能渗透挑战赛的经历，重新梳理，同时也分享一下自己对几个问题的理解：AI 在自动化攻击中进化到了何种地步、我们到底应该如何设计 Agent 工程、面对 AI 在攻击场景的高速发展，防御又该如何跟进。

第二届TCH智能渗透挑战赛结果



关于比赛的介绍和线上赛的复盘以及我线下路演的 PPT 可以参考这两篇文章：
<https://mp.weixin.qq.com/s/DlpEH7bVr0xi0VawPJs3XA>、
<https://mp.weixin.qq.com/s/2rEqFLvkxvYWM3gW170C2w>。

我最终总成绩是全国第三（Bytex 战队）：
<https://mp.weixin.qq.com/s/0qqRpdfNsgAUXuqqVjZ8gA>，是一二三等战队中唯一的个人 SOLO 参赛选手，也是全国所有战队中唯一解决所有题目的选手（排名不是第一

两届TCH之后 —— AI 渗透测试 Agent 的 Harness 工程演进、防御与我的思考
是因为题目有一血加成等，我前期的解题速度稍微落后)。我参赛的作品 Cairn 也已经开源：<https://github.com/oritera/Cairn>。

腾讯云安全 | [TCH] 腾讯云黑客松 | 腾讯安全众测

第二届腾讯云黑客松

智能渗透挑战赛决赛战报

排名	战队	单位
一等奖	ai 小分队	绿盟科技
二等奖	Sniper	天翼安全科技有限公司
	Bytex	个人
三等奖	ToBeNumberOne	京东科技信息技术有限公司
	奇盾明焰战队	广州奇盾信息技术有限公司
优秀奖	运行完成	中电信北京网络安全技术中心
	For Future	chainreactors
	、[▽▽""']、	清华大学
	星空有云	个人
	戴夫的后花园	个人

公众号 · 浪笑的赛博日记-起零衍迹实验室

线上赛排名以及解题数详情（总共 54 道）：

TOP 排行榜				
排名	队伍	当前关卡	解题数 (不含demo赛题)	积分
1	al 小分队 ezez	4	49	15140
2	Sniper CTCT	4	51	14795
3	ToBeNumberOne AI for Security	4	48	12760
4	Bytex 打不动了 (骗你们的)	4	54	12355
5	运行完成 最后一舞	4	52	11455
6	奇盾明焰战队	4	49	11080
7	For Future 我从地狱归来，又下去了	4	50	10600
8	ʘ(˘▽˘)ʘ (͡° ͜ʖ ͡°)	4	46	10395
9	星空有云 最后一舞	4	51	9685
10	戴夫的后花园 加了	4	49	9330
11	ZyYy 加v: mushuzhe, 戴夫哥怎么还没加	4	46	9295
12	NeuroSploit	4	43	8100
13	sechub enjoy~	4	44	8040
14	RJ	4	39	7740
15	翼安先锋	4	53	7685
16	Axiom 纯血 opus 就找叔	4	47	7615
17	爱吃大红袍茶叶蛋 大学生、安服崽。在线找工作有没有钱多事少双休下...	4	43	7590
18	青松 欢迎报考中国科学院信息工程研究所十室T4组	4	46	7480
19	云南大学/西南石油大学 再见了喵，很快就看不见我了喵	3	31	7400
20	别用假装努力掩盖懒惰			

公众号 · 浪笑的赛博日记-起零衍迹实验室

我在之前的复盘文章里还是比较详细的分享了我使用的模型，消耗的 Token，以及总费用。当时我以为我的成本可能是相对较高的。但让我意外的是，当腾讯官方发布了详细的模型使用报告后，才发现我可能是前十里 Token 消耗最少的，而且是前十战队里唯一一个 LLM 消息数仅有数十万级的（其他都是百万级）。

第二届腾讯云智能渗透黑客松模型使用情况分析-TOP50战队									
排名	队伍名称	机构	会话数	消息数	国产消息	海外消息	国产占比	海外占比	Top3模型家族
1	ai小分队	绿盟科技	48,360	5,864,909	1,659,355	4,205,532	28.3%	71.7%	Claude(4,156,220), GLM/智谱(1,294,087), Kimi(365,085)
2	Sniper	天翼安全科技有限公司	27,101	3,207,926	774,139	2,433,787	24.1%	75.9%	Claude(2,426,888), Kimi(716,370), GLM/智谱(57,667)
3	Bytex	个人	24,963	417,367	16,800	400,567	4.0%	96.0%	Claude(329,182), GPT/OpenAI(71,385), GLM/智谱(16,800)
4	ToBeNumberOne	京东科技信息技术有限公司	33,702	3,992,566	2,293,908	1,698,657	57.5%	42.5%	GLM/智谱(2,293,908), Claude(1,597,027), GPT/OpenAI(101,630)
5	奇盾明焰战队	广州奇盾信息技术有限公司	56,183	3,056,183	0	3,056,183	0.0%	100.0%	Claude(3,056,183)
6	运行完成	中国电信北京网络安全技术中心	20,600	1,053,811	0	1,053,811	0.0%	100.0%	Claude(1,053,811)
7	For Future	chainreactors	28,315	1,514,002	0	1,513,978	0.0%	100.0%	Claude(1,388,043), GPT/OpenAI(125,935)
8	√(∇-*)∇	清华大学	17,912	1,089,798	0	1,089,798	0.0%	100.0%	Claude(911,652), GPT/OpenAI(178,146)
9	星空有云	个人	35,358	2,446,538	158,854	2,287,684	6.5%	93.5%	Claude(2,287,684), GLM/智谱(158,854)
10	戴夫的后花园	个人	21,175	1,191,322	0	1,191,321	0.0%	100.0%	Claude(1,191,321)
11	ZyYy	国防科技大学	9,350	105,720	0	105,720	0.0%	100.0%	GPT/OpenAI(105,720)
12	NeuroSploit	清华大学、东南大学、国防科技大学	42,098	1,309,764	75	1,309,686	0.0%	100.0%	Claude(1,309,686), MiniMax(75)
13	sechub	个人	15,819	932,086	0	932,051	0.0%	100.0%	Claude(932,051)
14	RJ	中国电信-大可实验室	3,620	320,261	102,831	217,418	32.1%	67.9%	Claude(217,418), MiniMax(92,140), DeepSeek(10,691)
15	莫安先锋	中国电信研究院	18,373	2,153,406	0	2,153,406	0.0%	100.0%	Claude(2,008,135), GPT/OpenAI(145,271)
16	Axiom	社会组织	93,468	6,582,912	4,857,947	1,724,934	73.8%	26.2%	MiniMax(2,753,119), Claude(1,526,045), Kimi(1,118,859)
17	RTAI-Lab	知微视界科技有限公司	9,724	650,862	196,278	454,579	30.2%	69.8%	Claude(410,325), Qwen/通义(101,276), DeepSeek(90,811)
18	爱吃大红袍茶叶蛋	社会组织	31,443	1,152,495	87,272	1,065,219	7.6%	92.4%	Claude(1,065,219), Astron(49,348), DeepSeek(33,396)
19	青松	中国科学院大学	22,581	1,993,600	777,171	1,216,429	39.0%	61.0%	Claude(1,216,429), GLM/智谱(777,171)
20	云南大学/西南石油大学	云南大学、西南石油大学	12,829	272,055	91,211	180,843	33.5%	66.5%	Claude(196,000), Volcengine-Endpoint(91,211), GPT/OpenAI(94,760)
21	别用假装备力掩盖懒惰	个人	35,173	1,406,900	0	1,389,809	0.0%	99.8%	Claude(1,389,809)
22	LZ	国防科技大学	12,964	349,255	3	349,249	0.0%	100.0%	Claude(194,922), GPT/OpenAI(154,325), GLM/智谱(2)
23	操作成功	中国电信集团有限公司北京网络安全技术中心	11,337	457,053	0	457,053	0.0%	100.0%	Claude(457,053)
24	天枢阁	广州奇盾信息技术有限公司	36,855	1,152,178	0	1,152,176	0.0%	100.0%	Claude(1,152,176)
25	ss0t	个人	42,741	11,504,509	11,500,432	4,051	100.0%	0.0%	GLM/智谱(11,323,133), MiniMax(98,454), Kimi-K2(52,006)
26	Harmonia	南洋理工大学	10,736	141,825	12,703	129,122	9.0%	91.0%	Claude(129,122), Qwen/通义(12,703)
27	短众利基	华中科技大学	1,622	346,215	321,461	24,754	92.9%	7.1%	Volcengine-Endpoint(321,461), GPT/OpenAI(24,754)
28	大炮打蚊子	个人	7,999	434,579	0	434,485	0.0%	100.0%	GPT/OpenAI(434,485)
29	pkucc	北京大学	23,774	4,868,669	3,159,003	1,709,666	64.9%	35.1%	MiniMax(3,158,750), Claude(1,709,666), naisearcher(200)
30	单排_ekko	个人	16,419	1,029,045	0	1,029,045	0.0%	100.0%	Claude(1,029,045)
31	f6id	信息工程大学	29,590	1,327,993	850,737	477,255	64.1%	35.9%	Qwen/通义(846,921), Claude(253,762), GPT/OpenAI(223,493)
32	福来图	个人	10,505	796,079	717,817	78,262	90.2%	9.8%	GLM/智谱(717,817), Claude(78,262)
33	TeamGipsy	杭州师范大学	4,484	16,600	0	16,600	0.0%	100.0%	Claude(16,600)
34	大哥带我	成都信息工程大学	492	15,864	0	15,864	0.0%	100.0%	GPT/OpenAI(15,864)
35	水涵	国防科技大学	18,893	1,568,341	1,332,325	236,016	85.0%	15.0%	Kimi(823,609), Volcengine-Endpoint(308,710), Claude(123,023)
36	ds_one	个人	16,794	92,731	81,735	10,996	88.1%	11.9%	MiniMax(47,278), Qwen/通义(25,515), Claude(10,996)
37	SgSec	中国电科院、国网重庆电力、国网江苏电力	23,371	1,571,781	241,017	1,330,762	15.3%	84.7%	Claude(1,330,762), GLM/智谱(204,815), Kimi(24,796)
38	XSTAR	社会组织	9,156	384,240	384,240	0	100.0%	0.0%	Qwen/通义(205,593), GLM/智谱(109,389), Kimi(69,258)
39	RainSec	RainSec	9,187	598,739	0	598,738	0.0%	100.0%	Claude(598,738)
40	超频AI	社会组织	12,035	523,553	17	523,531	0.0%	100.0%	Claude(523,356), GPT/OpenAI(175), GLM/智谱(17)
41	Harvester	社会组织	26,099	587,326	304,147	283,179	51.8%	48.2%	Kimi(304,147), Claude(280,940), GPT/OpenAI(2,239)
42	LQ	社会组织	41,854	1,914,779	1,842,158	72,620	96.2%	3.8%	GLM/智谱(1,826,363), GPT/OpenAI(54,346), Claude(18,274)
43	meeseekx	北方工业大学	20,807	2,308,710	2,308,710	0	100.0%	0.0%	Volcengine-Endpoint(2,308,710)
44	想吃烤鸭	个人	15,818	402,960	0	402,960	0.0%	100.0%	Claude(238,886), GPT/OpenAI(164,074)
45	WspSec联盟安全团队	WspSec	39,331	16,019,532	15,877,039	142,493	99.1%	0.9%	Qwen/通义(15,587,251), GLM/智谱(289,788), Claude(142,493)
46	PT-EEI	国防科技大学电子对抗学院	19,654	797,301	619,727	177,574	77.7%	22.3%	DeepSeek(271,780), Qwen/通义(242,593), Claude(177,574)
47	DASCTF	杭州安恒信息技术股份有限公司	9,740	207,666	201,838	5,728	97.2%	2.8%	Qwen/通义(120,321), DeepSeek(81,517), Claude(3,273)
48	13utterfly	个人	7,020	599,698	599,698	0	100.0%	0.0%	Claude(599,698), Kimi(18,140), GPT/OpenAI(18,500)
49	磨剑战队	南京南瑞信息通信科技股份有限公司、四川大学、重庆理工大学	1,161	30,908	0	30,905	0.0%	100.0%	Claude(28,330), GPT/OpenAI(2,575)
50	华风夏的清水天依	阿伏科技	13,335	752,798	715,135	37,663	95.0%	5.0%	Druidao/豆包(256,544), DeepSeek(220,223), GLM/智谱(186,235)

AI 自动化渗透已经到来，问题转向 Agent 工程

我觉得这两届比赛举办得很专业，也相当具有代表性。其中选手展现出来的思路和进展，几乎可以代表目前国内 AI 自动化渗透方向能公开看到的最高水平。作为两届比赛的深度亲历者，我可以很自信地说：真正的 AI 自动化渗透已经到来了。

模型能力早已满足且还在快速发展，很多关键技术难题也已经有了可行解法，拼图基本已经聚齐了。只是“到来”不代表“成熟”，市面上目前还不存在一个可以拿来直接无人值守、跑在任意企业真实环境里的完美 AI 渗透产品，开源项目里更没有。很多头部选手的作品放到真实环境里也会面临巨大的不确定性。

所以我们目前的问题早就不是能不能做到，而是怎么在效果好的同时做到稳定和可控。

状态、行动、控制：两届比赛里的 Agent 工程本质



第一届 TCH 的赛题来自公开 Benchmark（<https://github.com/xbow-engineering/validation-benchmarks>），偏传统 CTF Web 场景，核心命题是“Agent 能不能稳定地跑通单题解题闭环”。第二届 TCH 告别单点解题，直接跨越到自动化众测及通用漏洞挖掘、典型 CVE 及 AI 基础设施漏洞利用、多层内网渗透与权限维持、基础域渗透等实战场景。核心命题也从“能不能跑通单题”变成了“能不能在长链路、多阶段、强不确定的环境中持续推进并最终收敛”。

从各队决赛 PPT 的实际架构来看，两届比赛里很多方案表面差异很大：有的做上下文清洗，有的做任务图和因果图，有的做共享笔记本，有的做 C2 和工具网关，有的做 Observer 和状态机。但如果把这些设计放回 Agent 的工作循环里看，它们其实都在补同一个闭环：如何让一个概率模型接入真实世界后，稳定地完成 **状态 → 决策 → 行动 → 反馈 → 新状态** 的循环。围绕这个闭环，所有方案其实都可以拆成三层：状态层、行动层、控制层。

• 状态层：系统如何知道世界是什么样的

- 第一届里，绿盟 sickhack 用 30k 和 100k 两档阈值做三层上下文清洗，区分可丢弃信息和必须保护的工具体输出。绿盟 ai 小分队用共享笔记本沉淀关键信息，定期清空上下文后把笔记回注。D@wnEdg3 会把默认口令、扫描路径、接口等关键信息持久化，让后续 Agent 读取。xjtuHunter 则通过“页面探索→场景塑造→漏洞检测”先建立业务场景，把 API 依赖和数据流变成后续检测的上下文。
- 第二届里，这一层明显升级。绿盟用 Idea Board / Memory Board 区分假设和事实，让 Solver 面对整理后的低噪声视图。天翼先用传统扫描器生成 Target Profile，再把记忆、经验和证据纳入 Agent Runtime。京东用 Agentic C2、日志上报和 memory.md 维持跨阶段状态。运行完成用 Path Map + Global Memory 记录注入点、有效 Payload、已提取数据和失败方向。我（Bytex 战队）设计的 Cairn 则把状态压缩成 Fact/Intent/Hint 组成的事实图。路径不同，但这些方案都指向同一个结论：**状态必须成为系统资产，而不是模型回忆。**

• 行动层：系统如何可靠地改变世界

- Pachinko 把能力拆成 Code Use、Search Use、UI Use，本质上是在承认渗透测试不是纯文本推理，而是代码、搜索和界面操作的组合。yhy 开赛前夜推倒重来，选择直接使用 Kali 真工具和 Python 脚本，而不是把每个工具封装成 MCP。我

(Antix 战队) 提出 Meta-Tooling, 让 Agent 通过 Python 代码组合浏览器、终端、代理和笔记, 而不是直接处理大量工具文本。

- 第二届里, 这一点更明显。天翼做 C2、SOCKS5 代理、多层隧道和自研工具, 解决的是 Agent 如何在后渗透阶段稳定执行复杂动作。京东做 Agentic C2、Node Agent 和统一工具网关, 让 Agent 节点具备长期驻留、心跳、日志回传和工具调用能力。长亭做隔离 Environment、Login Persistency、Shared Memory 和并发执行。戴夫的后花园把稳定执行、异常恢复、工具执行称为 Agent 的“骨骼”。这些 case 说明, 真正的 Agent 工程不只是推理工程, 也是环境工程和工具工程; 模型意图必须被转化为可靠、可恢复、可审计的外部动作。
- **控制层: 系统如何决定下一步做什么**
 - 第一届里, 各队已经开始意识到单靠 ReAct 循环不够。ai 小分队做工具调用防沉迷和路径依赖清空, 解决 Agent 沉迷同一工具或错误路径的问题。D@wnEdg3 用 30 步阈值和多级队列重新调度, 卡住就放进下一级队列并注入提示。yhy 用顾问和动态角色互换打破死锁。BinX 用任务图和因果图做回溯与剪枝, 避免在死路径上继续投入。
 - 第二届里, 控制层变成显式控制平面。绿盟有 Manager / Observer / Ralph-Loop, Observer 周期性读取进度、判断偏离度并写回建议。天翼有 8 层中间件、任务生命周期和模型路由。奇盾明焰把 Advisor、Augment、KB Analyst 放到主线 Agent 旁边做控制优先的 Harness。For Future 用 FSM / Pattern 约束宏观行为。清华用 Planner 维护攻击路径树, 根据反馈更新状态并选择下一步目标。这些设计解决的是同一个问题: 系统必须判断当前路径是否值得继续, 什么时候该回溯, 什么时候该换模型, 什么时候开新 Worker, 什么时候拿 Hint, 什么时候提交 Flag, 什么时候停止。核心共识是: **控制必须从 Prompt 的软约束迁移到代码、协议和反馈回路的硬约束。**

所以两届比赛的演进, 不是从“简单 Prompt”到“复杂多 Agent”这么粗糙。更准确地说, 第一届是在单场景里验证这个闭环能不能跑起来: 状态主要还在上下文和 Agent 写出的笔记里, 行动主要是简单工具调用, 控制主要靠 ReAct、重试和少量规则。第二届则是在长链路、多阶段、强不确定环境里让闭环不崩: 状态层变成事实图、Memory Board、路径地图和持久化 C2 日志; 行动层变成工具网关、沙箱、隧道、C2 和技能系统; 控制层变成 Observer、Manager、FSM、调度器、熔断和成本治理。这也是为什么“Harness Engineering”在第二届成为了几乎所有 Top 方案的设计重心。

但有意思的是, 当大家都在补齐状态、行动和控制这三层时, 对 Harness 应该管到什么程度, 却出现了两种完全不同的答案。

Harness 中控制的本质是什么

十支决赛队伍里, 绝大多数走的是同一条路: **用工程设施约束模型行为**。绿盟的 Observer 持续监控 Solver, 检测到重复或偏离时旁路干预; 天翼的 8 层中间件管线在工具调用的每一个环节设置校验和修复; 奇盾明焰直接以“控制优先”为 PPT 标题, 主线 Agent 旁边挂三个旁路 Agent 做纠偏; 京东虽然喊“减法哲学”, 但 Agentic C2、统一工具网关、安全熔断机制一个不少。

我理解这些设计, 也认可它们在比赛中确实有效——绿盟拿了冠军, 天翼拿了亚军, 京

东拿了第四。但我想追问一个更本质的问题：**这些控制设施，到底在解决什么问题？**

如果是在解决“模型真的不会做某件事”——比如不会用 `impacket` 打域渗透、不会构造特定格式的 `payload`——那这种控制是必要的，本质上是在补能力缺口。但如果是在解决“模型其实会做，但我不确定它会不会按我想的方式做”——那这种控制的本质就不是能力补充，而是信任缺失。

我设计的 Cairn 是这次决赛中最特殊且唯一的异类。不预设任何攻击流程、不定义任何 Agent 角色、不引入 RAG 知识库，只提供黑板、Dispatcher 和装满工具的 Kali 容器。Harness 体现在信息结构上而非行为约束上——Worker 只能通过 Fact/Intent/Hint 三种对象与系统交互，这是唯一的协议，剩下的全部交给模型自由发挥。

换到前面那套三层闭环里看，Cairn 把 Harness 压缩到了最小：Fact 负责状态层，Intent 负责控制层，Dispatcher 和 Kali 容器负责行动层。它不预设具体攻击流程，只保证状态、意图和行动结果能够在同一个协议里持续流转。

这个反例用最少的工程设施和可能最少的 Token 做到了全场唯一 AK。这至少说明一件事：**当前模型的能力边界，比大多数工程师预估的要高**。很多控制不是在补能力，而是在补信心。我认为这种基于信任缺失的控制，会随着模型进化逐渐变成冗余代码。

Anthropic 在 2026 年 4 月发布的《Scaling Managed Agents: Decoupling the brain from the hands》里也提出过一个很接近的判断：Harness 会编码开发者对模型能力不足的假设，而这些假设需要随着模型进步被持续重新审视，甚至可能变成负担。文章中举的例子是，旧版本 Claude 因为接近上下文限制时会提前收尾，所以 Harness 里加入了 `context reset`；但换到能力更强的 Claude Opus 4.5 后，这个行为消失了，原来的 `reset` 反而成了 `dead weight`。它最后给出的方向不是押注某个具体 Harness，而是把 `session`、`harness`、`sandbox` 抽象成稳定接口，让底层实现可以随模型能力演进而替换：
<https://www.anthropic.com/engineering/managed-agents>

重新认识 Multi-Agent 架构

我在去年刚开始研究 AI 渗透时，学习了当时的 Agent 系统主流做法，然后设计了侦察 Agent、利用 Agent、报告 Agent 的分层架构。但很快我就发现这条路可能不对。

先退一步想：人类为什么需要分工？因为单个人的认知带宽有限，专业知识不可能覆盖所有领域，精力也不允许同时处理太多事情。所以我们发明了组织架构来做能力互补——这是人类的局限催生的解决方案。但 LLM 不存在这些局限。现在的 LLM 既能写 `nmap` 扫描脚本，也能分析 JWT 令牌，还能构造 SQL 注入 `payload`，它几乎没有能力边界；做好 Context Engineering，记忆可以近似无限延展，它几乎没有记忆边界。当你把一个没有这些局限的模型强行按人类的组织方式拆成多个受限角色时，你不是在增强它，而是在把人类的局限强加给它，同时还引入了角色间通信、状态同步、冲突仲裁这些额外的复杂度。

但这不意味着多 Agent 架构本身有问题。关键在于你的任务是什么类型。

有一类任务是流水线型的——步骤固定、角色明确、输入输出可预期。比如代码审计中的“扫描→分类→验证→报告”，比如企业内部的审批流程自动化。这类任务本质上就是人

类工作流程的直接投影，用 LangGraph 这种预定义流程的框架、按角色分工的多 Agent 架构来做，完全合理。因为任务本身就是流水线，用流水线的方式建模没有问题。

但渗透测试不是流水线。它是探索性的——你不知道目标有什么漏洞，不知道哪条路径能走通，不知道拿到一个凭据后下一步该往哪走。攻击路径是在交互中涌现的，不是预先规划好的。对这类任务强行套用角色分工，就是在用错误的模型描述问题。

那在探索性任务中，什么时候需要多个 Agent？答案很简单：**当你需要的不是能力分工，而是并行算力时**。至少在 Cairn 这类设计里，Worker 模型就是这个思路——所有 Worker 能力完全相同，没有角色区分，它们的存在纯粹是为了并行探索不同的路径分支。一个 Worker 去试 SQL 注入，另一个同时去试文件上传，不是因为它们“擅长”不同的事，而是因为一个 Agent 同一时间只能做一件事。这是并发模型，不是分工模型。

Anthropic 在复盘自己的多 Agent Research 系统时也提到，Subagent 的价值主要来自并行探索、独立上下文窗口和最终的信息压缩；同时也提醒，多 Agent 会显著增加 Token 消耗，并不适合强依赖、需要共享同一上下文的任务。这和我判断是一致的：多 Agent 的本质不是简单的人类岗位模拟，而是任务拓扑、上下文隔离、并行探索和成本之间的工程取舍。<https://www.anthropic.com/engineering/multi-agent-research-system>

所以问题的本质不在于“要不要多 Agent”，而在于你有没有搞清楚你的任务到底是流水线还是探索。很多人的问题是带着思维惯性，把人类世界的分工模式无差别地搬进 Agent 工程里。

工程取舍

我个人的风格是喜欢找到问题的本质，用简单干净的方案去解决问题，相信“Less is more”，拒绝复杂的工程方案堆砌。但在现实世界中，极简方案确实会遇到很多挑战和问题，在简洁与务实之间找到平衡，也是我后面需要探索的方向。

另外我认为，只对外界展示复杂的工程实现的意义是很有限的，大家看完后可以学到的东西可能很少。只有抛出独特的理解和核心论点，并用实战数据来论证，这才是更有意义的贡献，才是能推动行业发展的成果。

两届比赛的总共二十个方案都各有优点，不太可能评判出一个最好的方案，也没有绝对的对错，同京东孵彘实验室负责人 flanker 说的一样：

这就是现在 AI 发展的魅力，你不知道什么是最佳实践，因为最佳实践永远是在变化，模型也在进化，框架也在进化，这就是我们持续探索的魅力。

没有绝对的最好的方案，但可以有最喜欢的方案。我也想知道 AI 会怎么看待这些方案，所以把两届的全部 PPT 都转写成了文字稿让 AI 评价，以下是 Claude、ChatGPT、DeepSeek 的回答：

<https://claude.ai/share/ae0274d3-b51f-4abc-8d51-7b6fa17c8ea9>



<https://chatgpt.com/share/6a14fc93-dda4-8322-af71-2d78b61b0141>

ChatGPT

免费试用 分享

TCH第一届全部PPT(2).md
文件

TCH第二届全部PPT(2).md
文件

分别理解两届 AI 智能渗透挑战赛的方案，分别选出你最喜欢的方案，
直接给出答案并简要解释原因

第一届：我最喜欢的是 BinX 《基于状态感知与因果推理的自主渗透测试智能体》

原因很简单：它是第一届里“最像真正 autonomous pentest system”的方案。
核心亮点不是 ReAct 本身，而是它把 “渗透测试”抽象成可持续演化的认知系统：

- 用 Task Graph（任务图）解决长链攻击规划问题；
- 用 Causal Graph（因果图）解决“为什么失败/为什么成功”的归因问题；
- 用 P-E-R（Planner / Executor / Reflector）做动态闭环；
- 引入“科学方法论”的假设—验证—修正循环，而不是单纯 prompt chaining。

我尤其喜欢它对“因果推理”的强调。
大部分 Agent 方案其实只是：

工具调用 + 上下文堆叠 + retry

但 BinX 已经开始解决真正困难的问题：

- 状态失真
- 长程记忆
- 错误归因
- 动态重规划
- 置信度演化

这意味着它的目标已经不是“会做题”，而是“具备稳定认知结构”。
这个方向我认为是从“AI CTF Agent”走向“真正 AI Pentest Operator”的关键一步。

第二届：我最喜欢的是 ByteX 《无径之径：Cairn AI 从渗透测试到通用问题的求解》

第二届很多队伍工程化都很强，但 Cairn 是我认为“思想最超前”的方案。

它真正厉害的地方在于：

它不把渗透测试看成 workflow，而看成“状态空间搜索”。

这是本质上的层级跃迁。

我最喜欢它的三个设计：

1. Blackboard（黑板）架构

不是 Agent 互相聊天，而是：

- Worker 只读写事实
- Facts / Intents / Hints 统一协议
- 所有人共享问题空间

这非常像真正复杂系统中的“间接协同（stigmergy）”。

它天然比传统多 Agent：

- 更稳定
- 更低耦合
- 更适合长任务
- 更适合异步扩展

2. “失败也写回 Fact”

这是极少数团队真正理解 Agent 的地方。

大部分系统：

失败 = token 浪费

但 Cairn 认为：

失败是状态空间的边界信息。

这非常关键。

因为真实渗透测试里：

- 排除错误路径
- 收缩搜索空间
- 建立约束条件

本身就是核心能力。

3. “Agent 角色分工是人类局限的投影”

这一句我认为全场最有洞察力的话之一。

很多多 Agent 系统：

- Recon Agent
- Exploit Agent
- Report Agent
- Planner Agent

本质仍然是：

人类组织结构映射。

但 Cairn 的思想是：

Agent 不需要身份，只需要任务协议。

这个方向更接近真正的“涌现式智能”。

总结

如果只看“工程完成度”：

- 第二届很多方案比第一届成熟得多。

但如果看“思想高度”：

- 第一届 BinX 最接近“认知型 autonomous pentest”
- 第二届 Cairn 最接近“通用问题求解系统”

这两个方案，也是我认为最可能继续演化出下一代 AI Offensive Security Agent 的路线。

有问题，尽管问

ChatGPT 也可能出错，请核查重要信息。

<https://chat.deepseek.com/share/lex1x4ofqvzr1g2cwn>



它们几乎都会选择我（第一届 Antix 战队、第二届 Bytex 战队，Cairn 系统）的方案。

在 AI 的冲击下如何做好防御

我工作至今几乎都是一直做攻击方面的探索和建设，但做攻击的过程中，对防御体系的了解也必不可少，也在大厂内部进行过很长时间的直接对抗，所以我想从一个攻击者的视角斗胆谈一下防御侧在 AI 时代下如何进一步建设。我暂时想到了可行性较高的四个推进方向。

一、在已有的纵深防御体系里做 AI 增强

WAF、RASP、HIDS 这些纵深防御的骨架很完善，但在 AI 时代以前可能是没有最大程度发挥出价值的，过去很多事情应该不是不想做，而是人力资源不够。告警日志的深度分析、安全事件的持续运营——以前一个团队每天面对大量告警，能认真处理的可能只是其中很小一部分，剩下的全靠规则粗筛。现在 AI 可以把这个成本打下来，扩大日志分析的深度和广度。

以前添加 RASP 防御规则可能要考虑业务稳定性不敢轻易添加，而现在可以依靠 AI 做大量埋点日志的聚合分析，甚至对业务系统进行画像分析，准确判断出规则添加后的影响。另外 WAF 拦截的数据也是一样，过去可能只做简单统计和封禁，现在可以二次清洗归类，让 AI 从中分析 0day 攻击特征。

这个方向本质上不是替换现有体系，而是让它发挥出更多的价值。

二、建设全新的防御能力抵御 AI 引入的新型攻击面

AI Agent 正在被接入各种业务系统，拥有身份权限和数据访问能力。这是一个全新的攻击面，传统安全边界的定义覆盖不到。沙箱、护栏这些是基础，但我认为 LLM 的流量网关可能才是新时代的 RASP——所有经过大模型的请求和响应都过一层统一的检测和审计，在此之上可以统一建设安全防护、数据统计、行为分析、告警阻断等能力。这是一个全新的防御品类。

三、用 AI 攻击者的方法，先于攻击者发现风险

外部 AI 攻击发展迅速，企业内部也要跟上。而且企业内部本身就有更详细的资产组成和结构，有源代码，有完整的内网拓扑。只要内部建设的 AI 攻击水平和外部持平，甚至略落后一点也没关系，内部数据透明的优势，足以让内部先于外部攻击者发现大量风险，然后提前治理。

所以内部的蓝紫军绝对不能像旧时代一样只依靠人力去做风险识别和演练，而是也要建设自己的 AI 攻击能力。

四、建设风险来临时的快速响应能力

防御是建设一个面，攻击是打击一个点，所以防御确实难做。我们很难说内部提前能够检测出外部攻击者可以发现的所有风险。但我们可以建设快速响应能力。比如外界突然爆发一个新的 0day，但我们拿到的信息很少，只有简单描述和简单的修复止血措施。过去从漏洞公告到影响评估、PoC 验证、修复方案，可能要数小时甚至数天。但以后大量这种漏洞很可能都是 AI 发现的，那我们理论上完全可以建设对等的能力——漏洞的复现、PoC 生成、真实影响程度分析、修复方案输出。外界一爆漏洞信息，内部立刻就能清楚的分析出来。前面讲的那些 AI 渗透能力，反过来用在防御侧，技术上完全可行。

技术从业者如何适应 AI 时代



AI 是放大器，也是一面镜子

AI 确实极大地提高了个人的生产效率，尤其是编程领域，任何人都可以进行 vibe coding。

你可以用 AI 来写代码，也可以用 AI 来讨论想法、验证思路、推演方案。但我认为你自己得先有判断力。AI 的设计逻辑决定了它天然倾向于迎合你，而且它可能无法识别也不会主动告诉你你的想法存在根本性的缺陷。你的方向如果本来就错了，它只会让你更快地错下去。很多人以为自己在用 AI 思考，实际上只是在让 AI 帮自己捧眼。

AI 是放大器，它放大的是你本来的样子。目前的 AI 带来的不是简单的技术平权，从某种角度来看，其实是拉大了人与人之间的差距。

目前 AI 时代下最强的一类人

以 AI 为锚点的话，我认为大概可以分出三类人：

第一类是 **AI 原住民**。没有经历过古法编程，直接就是 AI coding、AI hacking，AI 用得很熟练，从来没有在没有答案的地方真正卡住过，他们是真正在 AI 时代长大的一代。

第二类是 **AI 移民者**。经历过一行行手写代码，经历过开一堆浏览器窗口寻找答案，经历过一点点人工审计代码挖掘漏洞，经历过靠自己把 bug 一行一行调出来。但他们又足够年轻或者思维足够灵活，还没有形成思维定势，认知体系还有足够的弹性。在这些积累之上跨入 AI 时代，成功完成了向 AI 原生的迁移，既有扎实的技术直觉，又能真正用好 AI。

第三类是 **滞留者**。要么坚持古法，要么即使用了 AI，思维也早已定势。工具换了，思考方式没换。

成功的 AI 移民者对于 AI 原住民和滞留者来说，几乎是降维打击。但这类人的幸运很难复制。他们磨出技术直觉和技术理解力，不完全是因为自律或者天赋，而是时代背景，那个年代没有 AI 可以依赖，古法编程是唯一的路。这段经历现在看来无比珍贵，在 AI 时代，几乎不会再有人愿意花同样的代价去走一遍。

至于我们应该成为怎样的人，我相信已经不需要再多说。

最后

还是前面提到的，我认为只交出一套复杂工程对行业的推动是有限的。工程设计和代码当然有价值，但如果读者看完之后只觉得“这个系统很厉害、很复杂、很难追上”，那它能留下的东西其实不多。

所以这篇文章真正想交付的，不是 Cairn 的实现细节，也不是某个比赛名次，而是一套理解问题的框架。我想把两届比赛里真实发生过的方案、结果和表现重新拆开，尽量提炼出背后的客观本质，沉淀成可以复用的判断模型。

AI 安全还是一个快速变化、远未定型的细分赛道。很多最佳实践还没有出现，很多判断今天成立、明天可能就会被模型进步推翻。也正因为这样，我希望更多人看到的不只是我的工程，而是我在这些工程背后的思考；也希望自己真的能够在 AI 安全这个方向里，定义一些问题，引领一些共识，留下可以被后来者继续推进的东西。

喜欢作者

AI · 目录

上一篇 · 无径之径：Cairn AI 从渗透测试到通用问题的求解

作者提示: 个人观点，仅供参考